

# Bài 1: Tìm hiểu chuyên sâu về các cơ chế Attention trong các mô hình ngôn ngữ lớn (LLMs)

## Mục tiêu tổng quát

Bài viết phân tích chuyên sâu bốn cơ chế attention hiện đại: **Self-Attention**, **Multi-Query Attention (MQA)**, **Grouped-Query Attention (GQA)**, và **FlashAttention**, với:

- **Lý thuyết chi tiết:** Cơ chế hoạt động, độ phức tạp tính toán, ưu điểm, hạn chế, so sánh với Self-Attention gốc, ứng dụng trong LLMs, và mục tiêu cuối cùng trong thực tiễn.
- **Ví dụ thực tiễn:** Minh họa ứng dụng dễ hiểu, có tính thực tiễn cao.
- **Mã nguồn minh họa:** Triển khai bằng PyTorch, kèm ví dụ và trực quan hóa ma trận attention (được cung cấp trong tài liệu bổ sung).

## 1 Lý thuyết chuyên sâu, ví dụ thực tiễn và mục tiêu cuối cùng

Chúng tôi chọn bốn cơ chế attention: **Self-Attention**, **Multi-Query Attention (MQA)**, **Grouped-Query Attention (GQA)**, và **FlashAttention**. Dưới đây là phân tích chi tiết, bao gồm các khía cạnh nâng cao (gradient flow, cache optimization, hardware impact), ví dụ thực tiễn, và mục tiêu cuối cùng trong thực tiễn.

### 1.1 Self-Attention

#### 1.1.1 Cơ chế hoạt động và ý tưởng chính

Self-Attention, được giới thiệu trong bài báo *Attention is All You Need* (Vaswani et al., 2017), là nền tảng của kiến trúc Transformer. Nó cho phép mỗi token trong chuỗi đầu vào tương tác với tất cả token khác, nắm bắt mối quan hệ ngữ cảnh toàn cục mà không phụ thuộc vào thứ tự tuần tự như RNN.

- **Quy trình:**

- Với đầu vào  $X \in \mathbb{R}^{n \times d}$  ( $n$ : độ dài chuỗi,  $d$ : chiều embedding), Self-Attention tính toán:

$$Q = XW_Q,$$

$$K = XW_K,$$

$$V = XW_V,$$

với  $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ ,  $d_k = d/h$  ( $h$ : số head).

- Điểm attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

trong đó  $\sqrt{d_k}$  chuẩn hóa để ổn định gradient, và  $QK^T \in \mathbb{R}^{n \times n}$  là ma trận tương quan.

- Trong **Multi-Head Self-Attention (MHSA)**, quá trình được thực hiện trên  $h$  head song song, kết quả được nối và chiếu:

$$\text{MHSA}(X) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W_O,$$

với  $W_O \in \mathbb{R}^{d \times d}$ .

- **Chi tiết nâng cao:**

- **Scaling factor:**  $\sqrt{d_k}$  giữ phương sai của  $QK^T$  ổn định, đặc biệt khi  $d_k$  lớn (128 hoặc 256 trong LLMs).
- **Content-based addressing:** Self-Attention học cách phân bổ trọng số dựa trên nội dung, khác với CNN/RNN dựa vào vị trí cố định.
- **Gradient flow:** Phép nhân ma trận và softmax tạo gradient phức tạp, có thể gây vanishing/exploding gradient nếu không dùng LayerNorm.

### 1.1.2 Độ phức tạp tính toán

- **Thời gian:**  $O(n^2 \cdot d)$ 
  - Tính  $QK^T$ :  $O(n^2 \cdot d_k)$  mỗi head, tổng  $O(n^2 \cdot d)$ .
  - Nhân với  $V$ :  $O(n^2 \cdot d_k)$  mỗi head, tổng  $O(n^2 \cdot d)$ .
- **Không gian:**  $O(n^2 + n \cdot d)$ 
  - Ma trận attention:  $O(n^2)$ .
  - $Q, K, V$ :  $O(n \cdot d)$  mỗi ma trận.
- **Tác động:** Với  $n = 4096$ , ma trận attention chiếm hàng GB, gây khó khăn cho chuỗi dài.

### 1.1.3 Ưu điểm

- Nắm bắt mọi mối quan hệ giữa token, bất kể khoảng cách.
- Song song hóa tốt trên GPU, vượt trội RNN.
- Linh hoạt (hỗ trợ causal/bidirectional attention).

### 1.1.4 Hạn chế

- Độ phức tạp  $O(n^2)$  không khả thi cho chuỗi dài ( $n > 10000$ ).
- Tốn bộ nhớ do ma trận attention lớn.
- Hiệu quả giảm với các mối quan hệ xa nếu không tối ưu.

### 1.1.5 So sánh với Self-Attention gốc

Self-Attention là cơ chế gốc, làm chuẩn để so sánh các biến thể khác.

### 1.1.6 Ứng dụng trong LLMs

- **BERT:** Bidirectional Self-Attention cho phân loại văn bản, trả lời câu hỏi.
- **GPT:** Causal Self-Attention cho sinh văn bản tự hồi quy.
- **T5:** Self-Attention trong encoder/decoder cho seq2seq.
- **Chi tiết mới:** Các mô hình như LLaMA, Mistral, Grok dựa trên Self-Attention, kết hợp tối ưu hóa như MQA, GQA.

### 1.1.7 Ví dụ thực tiễn: Hệ thống trả lời câu hỏi về tài liệu

- **Tình huống:** Một công ty muốn chatbot trả lời câu hỏi từ tài liệu nội bộ 500 từ về quy trình làm việc. Người dùng hỏi: “Quy trình kiểm tra chất lượng diễn ra khi nào?”
- **Cách hoạt động:**
  - Tài liệu chia thành token: [“Quy trình”, “kiểm tra”, “chất lượng”, “diễn ra”, “vào”, “thứ Sáu”, ...].
  - Self-Attention cho phép token “chất lượng” chú ý đến “kiểm tra” và “thứ Sáu”, dù cách xa nhau.
  - Kết quả: Chatbot trả lời: “Quy trình kiểm tra chất lượng diễn ra vào thứ Sáu.”
- **Tính dễ hiểu:** Self-Attention giống người đọc thông minh, liên kết từ khóa trong tài liệu để trả lời nhanh, như tìm kiếm trên Google.
- **Ứng dụng:** Chatbot doanh nghiệp, trợ lý AI như Grok, hệ thống tìm kiếm thông tin.

### 1.1.8 Mục tiêu cuối cùng trong thực tiễn

- **Mục tiêu:** *Tạo ra các biểu diễn ngữ cảnh phong phú và chính xác cho chuỗi đầu vào*, giúp LLMs hiểu sâu sắc mối quan hệ giữa các token, từ đó hỗ trợ các tác vụ phức tạp như trả lời câu hỏi, dịch máy, hoặc tóm tắt văn bản với chất lượng cao nhất.
- **Ý nghĩa:** Self-Attention hướng đến việc tối đa hóa khả năng biểu đạt của mô hình, đặc biệt trong huấn luyện, để xây dựng các hệ thống AI có khả năng hiểu và xử lý ngôn ngữ tự nhiên như con người, ví dụ: trả lời chính xác câu hỏi từ tài liệu dài hoặc sinh văn bản mạch lạc.

## 1.2 Multi-Query Attention (MQA)

### 1.2.1 Cơ chế hoạt động và ý tưởng chính

MQA, tối ưu cho suy luận trong LLMs, sử dụng nhiều Query head nhưng chỉ một Key và Value chung, giảm bộ nhớ cache trong suy luận tự hồi quy.

- **Quy trình:**

- Với  $X \in \mathbb{R}^{n \times d}$ , tính toán:

$$\begin{aligned} Q_i &= XW_{Q_i}, \quad \text{với head } i \in \{1, \dots, h\}, \\ K &= XW_K, \\ V &= XW_V, \end{aligned}$$

với  $W_{Q_i}, W_K, W_V \in \mathbb{R}^{d \times d_k}$ .

- Attention mỗi head:

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K^T}{\sqrt{d_k}}\right) V.$$

- Kết quả:

$$\text{MQA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O.$$

- **Chi tiết nâng cao:**

- **Cache suy luận:** MQA giảm bộ nhớ cache từ  $O(h \cdot n \cdot d_k)$  xuống  $O(n \cdot d_k)$ , vì chỉ lưu một  $K, V$ .
- **Huấn luyện:** Chia sẻ  $K, V$  có thể làm giảm biểu đạt, cần điều chỉnh cẩn thận.
- **Tối ưu hóa:** Kết hợp với kernel fusion, mixed-precision để tăng tốc.

### 1.2.2 Độ phức tạp tính toán

- **Thời gian:**  $O(n^2 \cdot d)$ , tương tự Self-Attention.
- **Không gian:**
  - Ma trận attention:  $O(h \cdot n^2)$ .
  - $K, V$ :  $O(n \cdot d_k)$ , ít hơn  $O(h \cdot n \cdot d_k)$ .
  - Cache suy luận:  $O(n \cdot d_k)$ .
- **Tác động:** Với  $h = 32$ ,  $n = 4096$ , MQA giảm bộ nhớ cache khoảng 32 lần.

### 1.2.3 Ưu điểm

- Hiệu quả suy luận nhờ cache nhỏ.
- Giữ sức mạnh biểu đạt với nhiều Query head.
- Dễ tích hợp vào Transformer.

### 1.2.4 Hạn chế

- Giảm biểu đạt so với Self-Attention.
- Huấn luyện có thể kém ổn định.
- Vẫn  $O(n^2)$ , khó mở rộng chuỗi dài.

### 1.2.5 So sánh với Self-Attention gốc

- **Khác biệt:** MQA chia sẻ  $K, V$ , giảm bộ nhớ nhưng mất một số sắc thái ngữ cảnh.
- **Hiệu suất:** Nhanh hơn trong suy luận, nhưng chất lượng có thể thấp hơn.
- **Ứng dụng:** Tối ưu cho suy luận, Self-Attention tốt hơn cho huấn luyện.

### 1.2.6 Ứng dụng trong LLMs

- **Grok (xAI):** Tăng tốc suy luận cho chatbot.
- **Falcon:** Giảm chi phí suy luận.
- **Chi tiết mới:** Hữu ích trên thiết bị biên với bộ nhớ hạn chế.

### 1.2.7 Ví dụ thực tiễn: Chatbot sinh câu trả lời dài

- **Tình huống:** Bạn hỏi chatbot Grok: “Kể một câu chuyện về chú mèo.” Chatbot sinh: “Ngày xưa, có một chú mèo tên là Miu...” và tiếp tục đến 100 từ.
- **Cách hoạt động:**
  - Mỗi từ mới (như “Miu”) cần nhớ các từ trước.
  - MQA lưu một bộ  $K, V$ , giảm bộ nhớ so với Self-Attention (32 bộ).
  - Kết quả: Chatbot sinh câu chuyện mượt mà: “...Miu thích chạy nhảy trên cánh đồng.”
- **Tính dễ hiểu:** MQA như ghi chú ý chính của câu chuyện, giúp chatbot kể nhanh mà không tốn bộ nhớ, giống cách bạn tóm tắt khi kể chuyện.
- **Ứng dụng:** Chatbot thời gian thực, trợ lý AI trên điện thoại.

### 1.2.8 Mục tiêu cuối cùng trong thực tiễn

- **Mục tiêu:** Tối ưu hóa tốc độ suy luận của LLMs trong các ứng dụng thời gian thực, đặc biệt trong các hệ thống tự hồi quy như chatbot, bằng cách giảm đáng kể bộ nhớ cache Key-Value, từ đó cho phép triển khai trên thiết bị có tài nguyên hạn chế mà vẫn duy trì chất lượng tốt.
- **Ý nghĩa:** MQA nhắm đến việc làm cho các mô hình sinh văn bản (như Grok) hoạt động nhanh chóng và hiệu quả, ví dụ: sinh câu trả lời dài hoặc câu chuyện mà không gây độ trễ, ngay cả trên điện thoại hoặc máy tính yếu.

## 1.3 Grouped-Query Attention (GQA)

### 1.3.1 Cơ chế hoạt động và ý tưởng chính

GQA là sự thỏa hiệp giữa Self-Attention và MQA, chia Query head thành  $g$  nhóm, mỗi nhóm chia sẻ một Key và Value, cân bằng hiệu quả và chất lượng.

- **Quy trình:**
  - Với  $h$  head,  $g$  nhóm ( $g \leq h$ ), mỗi nhóm có  $h/g$  Query head.
  - Tính toán:

$$\begin{aligned}Q_{i,j} &= XW_{Q_{i,j}}, \quad \text{với head } i \text{ trong nhóm } j, \\K_j &= XW_{K_j}, \\V_j &= XW_{V_j}.\end{aligned}$$

- Attention mỗi head:

$$\text{head}_{i,j} = \text{softmax}\left(\frac{Q_{i,j}K_j^T}{\sqrt{d_k}}\right)V_j.$$

- Kết quả:

$$\text{GQA}(X) = \text{Concat}(\text{head}_{1,1}, \dots, \text{head}_{h/g,g})W_O.$$

- **Chi tiết nâng cao:**
  - **Số nhóm  $g$ :**  $g = 1$  là MQA,  $g = h$  là Self-Attention. Thường chọn  $g = 4, 8$ .
  - **Chất lượng:** GQA gần với Self-Attention khi  $g$  lớn.
  - **Cache:** Lưu  $g$  bộ  $K, V$ , tiết kiệm hơn Self-Attention.

### 1.3.2 Độ phức tạp tính toán

- **Thời gian:**  $O(n^2 \cdot d)$ .
- **Không gian:**
  - Ma trận attention:  $O(h \cdot n^2)$ .
  - $K, V$ :  $O(g \cdot n \cdot d_k)$ .
  - Cache suy luận:  $O(g \cdot n \cdot d_k)$ .
- **Tác động:** Với  $h = 32, g = 8$ , GQA giảm bộ nhớ cache khoảng 4 lần.

### 1.3.3 Ưu điểm

- Cân bằng chất lượng và hiệu quả.
- Linh hoạt với số nhóm  $g$ .
- Hiệu quả suy luận tốt.

#### 1.3.4 Hạn chế

- Vẫn  $O(n^2)$ .
- Phụ thuộc  $g$ :  $g$  nhỏ giảm chất lượng,  $g$  lớn mất hiệu quả.
- Triển khai phức tạp hơn MQA.

#### 1.3.5 So sánh với Self-Attention gốc

- **Khác biệt**: GQA giảm  $K, V$  từ  $h$  xuống  $g$ , tiết kiệm bộ nhớ.
- **Hiệu suất**: Gần Self-Attention về chất lượng, nhanh hơn trong suy luận.
- **Ứng dụng**: Phù hợp cả huấn luyện và suy luận.

#### 1.3.6 Ứng dụng trong LLMs

- **LLaMA 2**: Tối ưu hiệu suất nghiên cứu.
- **Mixtral**: Tăng tốc suy luận.
- **Grok**: Cải thiện xử lý nhanh.
- **Chi tiết mới**: Hữu ích trong dịch máy, tóm tắt văn bản dài.

#### 1.3.7 Ví dụ thực tiễn: Dịch máy thời gian thực

- **Tình huống**: Ứng dụng dịch thuật dịch câu: “The cat runs fast on the green field” sang tiếng Việt trên điện thoại.
- **Cách hoạt động**:
  - Token: [“The”, “cat”, “runs”, “fast”, “on”, “the”, “green”, “field”].
  - GQA chia 32 head thành 8 nhóm, mỗi nhóm 4 head, chia sẻ  $K, V$ , giảm bộ nhớ.
  - Nhóm 1 chú ý “cat”, nhóm 2 chú ý “runs”, đảm bảo bản dịch: “Con mèo chạy nhanh trên cánh đồng xanh.”
- **Tính dễ hiểu**: GQA như nhóm dịch giả chia sẻ tài liệu tham khảo, dịch nhanh và chính xác, giống Google Translate.
- **Ứng dụng**: Dịch máy, xử lý ngôn ngữ trên thiết bị di động.

#### 1.3.8 Mục tiêu cuối cùng trong thực tiễn

- **Mục tiêu**: *Cân bằng giữa chất lượng biểu đạt và hiệu quả tính toán*, cho phép LLMs đạt hiệu suất cao trong cả huấn luyện và suy luận, đặc biệt trong các ứng dụng yêu cầu xử lý nhanh và chính xác như dịch máy hoặc phân tích văn bản trên nhiều nền tảng.
- **Ý nghĩa**: GQA hướng đến việc triển khai LLMs linh hoạt, từ nghiên cứu (như LLaMA) đến ứng dụng thực tế (như dịch thuật trên điện thoại), đảm bảo chất lượng gần với Self-Attention nhưng với chi phí thấp hơn, ví dụ: dịch văn bản nhanh chóng mà vẫn giữ ngữ nghĩa.

## 1.4 FlashAttention

### 1.4.1 Cơ chế hoạt động và ý tưởng chính

FlashAttention (Dao et al., 2022) và FlashAttention-2 tối ưu Self-Attention bằng cách chia chuỗi thành khối, tính toán trong bộ nhớ nhanh GPU (SRAM), giảm chi phí bộ nhớ.

- **Quy trình:**
  - Chia  $Q, K, V$  thành khối (block size: 128-256).
  - Tính attention từng khối trong SRAM, không lưu ma trận  $n \times n$ .
  - Tích hợp nhân ma trận, softmax, nhân  $V$  trong một kernel GPU.
  - FlashAttention-2 cải tiến phân chia công việc, tăng sử dụng SM (streaming multiprocessor).
- **Chi tiết nâng cao:**
  - **Recomputation:** Tái tạo ma trận attention trong lan truyền ngược, giảm bộ nhớ gradient.
  - **Phần cứng:** Tận dụng SRAM nhanh hơn HBM hàng chục lần.
  - **Mở rộng:** Hỗ trợ chuỗi dài (64k token).

### 1.4.2 Độ phức tạp tính toán

- **Thời gian:**  $O(n^2 \cdot d)$ , nhưng nhanh hơn 2-4 lần thực tế.
- **Không gian:**  $O(n \cdot d + b^2)$ , với  $b$ : block size.
- **Tác động:** Với  $n = 16384$ , giảm bộ nhớ từ GB xuống MB.

### 1.4.3 Ưu điểm

- Nhanh hơn 2-4 lần, tiết kiệm bộ nhớ.
- Xử lý chuỗi dài (64k token).
- Tương thích với Transformer.

### 1.4.4 Hạn chế

- Yêu cầu GPU hiện đại (NVIDIA A100, H100).
- Khó tùy chỉnh cho biến thể attention khác.
- Triển khai phức tạp.

### 1.4.5 So sánh với Self-Attention gốc

- **Khác biệt:** Giữ toán học, tối ưu triển khai.
- **Hiệu suất:** Nhanh hơn, tiết kiệm bộ nhớ, phù hợp chuỗi dài.
- **Ứng dụng:** Cả huấn luyện và suy luận.

### 1.4.6 Ứng dụng trong LLMs

- **Hugging Face:** Tăng tốc LLaMA, GPT.
- **LLaMA 2, Grok:** Xử lý chuỗi dài.
- **Chi tiết mới:** Dùng trong vision transformers, xử lý tín hiệu.

#### 1.4.7 Ví dụ thực tiễn: Tóm tắt tài liệu dài

- **Tình huống:** Sinh viên tóm tắt bài báo 10.000 từ về biến đổi khí hậu trên laptop có GPU.
- **Cách hoạt động:**
  - Bài báo chia thành hàng nghìn token.
  - Self-Attention tạo ma trận lớn, tốn GB bộ nhớ.
  - FlashAttention chia thành khối 256 token, xử lý trong SRAM, tiết kiệm bộ nhớ.
  - Kết quả: Tóm tắt nhanh: “Bài báo thảo luận tác động biến đổi khí hậu và giải pháp giảm phát thải.”
- **Tính dễ hiểu:** FlashAttention như người đọc chia nhỏ tài liệu, xử lý từng phần, giống cách bạn tóm tắt sách.
- **Ứng dụng:** Tóm tắt tài liệu, phân tích mã nguồn, xử lý dữ liệu lớn.

#### 1.4.8 Mục tiêu cuối cùng trong thực tiễn

- **Mục tiêu:** *Tăng tốc độ và giảm chi phí bộ nhớ để xử lý chuỗi dài trong LLMs*, cho phép huấn luyện và suy luận hiệu quả trên phần cứng hiện đại, hỗ trợ các tác vụ yêu cầu ngữ cảnh lớn như tóm tắt tài liệu dài hoặc phân tích dữ liệu phức tạp.
- **Ý nghĩa:** FlashAttention nhắm đến việc mở rộng khả năng của LLMs, giúp xử lý các tác vụ lớn (như tóm tắt tài liệu 10.000 từ hoặc phân tích mã nguồn hàng triệu dòng) trên GPU thông thường, tiết kiệm tài nguyên và thời gian.

## 2 So sánh tổng quan

| Tiêu chí           | Self-Attention                          | MQA                                   | GQA                             | FlashAttention                       |
|--------------------|-----------------------------------------|---------------------------------------|---------------------------------|--------------------------------------|
| Thời gian          | $O(n^2 \cdot d)$                        | $O(n^2 \cdot d)$                      | $O(n^2 \cdot d)$                | $O(n^2 \cdot d)$ (nhanh hơn thực tế) |
| Không gian         | $O(n^2 + n \cdot d)$                    | $O(n^2 + n \cdot d_k)$                | $O(n^2 + g \cdot n \cdot d_k)$  | $O(n \cdot d + b^2)$                 |
| Cache suy luận     | $O(h \cdot n \cdot d_k)$                | $O(n \cdot d_k)$                      | $O(g \cdot n \cdot d_k)$        | $O(n \cdot d + b^2)$                 |
| Ưu điểm            | Biểu đạt cao, song song tốt             | Hiệu quả suy luận, cache nhỏ          | Cân bằng chất lượng và hiệu quả | Nhanh, tiết kiệm bộ nhớ, chuỗi dài   |
| Hạn chế            | Tốn bộ nhớ, $O(n^2)$                    | Giảm biểu đạt, khó huấn luyện         | Phụ thuộc $g$ , $O(n^2)$        | Yêu cầu GPU, khó tùy chỉnh           |
| Ứng dụng           | BERT, GPT, T5                           | Grok, Falcon                          | LLaMA 2, Mistral                | LLaMA, Hugging Face, Grok            |
| Ví dụ thực tiễn    | Trả lời câu hỏi tài liệu                | Chatbot sinh câu chuyện               | Dịch máy thời gian thực         | Tóm tắt tài liệu dài                 |
| Mục tiêu cuối cùng | Biểu diễn ngữ cảnh phong phú, chính xác | Tối ưu tốc độ suy luận thời gian thực | Cân bằng chất lượng và hiệu quả | Tăng tốc, giảm bộ nhớ cho chuỗi dài  |

Bảng 1: So sánh tổng quan các cơ chế Attention

## 3 Tổng hợp

- Bài viết lý thuyết:



- **Định dạng:** PDF/Word.
- **Nội dung:**
  - \* Lý thuyết chi tiết: Cơ chế, độ phức tạp, ưu/hạn chế, so sánh, ứng dụng, mục tiêu cuối cùng.
  - \* Ví dụ thực tiễn: Trả lời câu hỏi, sinh câu chuyện, dịch máy, tóm tắt tài liệu.
  - \* Mục tiêu cuối cùng: Biểu diễn ngữ cảnh (Self-Attention), tốc độ suy luận (MQA), cân bằng chất lượng/hiệu quả (GQA), xử lý chuỗi dài (FlashAttention).
  - \* Bảng so sánh với cột “Ví dụ thực tiễn” và “Mục tiêu cuối cùng”.
  - \* Hình minh họa: Sơ đồ kiến trúc Self-Attention, MQA, GQA; quy trình block-based của FlashAttention.
  - \* Thảo luận: Tác động đến LLMs (Grok, LLaMA, Mixtral), xu hướng tương lai (sparse/linear attention).
- **Độ dài:** Khoảng 20-25 trang.
- **Notebook minh họa:**
  - **Định dạng:** Jupyter Notebook trên Colab.
  - **Nội dung:**
    - \* Mã nguồn Self-Attention, MQA, GQA, mã giả FlashAttention (được cung cấp trong tài liệu bổ sung).
    - \* Ví dụ thực tế với token từ các tình huống (tài liệu, câu chuyện, dịch máy, tóm tắt).
    - \* Trực quan hóa ma trận attention bằng heatmap, chú thích token.
    - \* Ô mã tương tác để thử nghiệm tham số (num\_heads, num\_groups, seq\_len).
  - **Tính tương tác:** Cho phép nhập chuỗi văn bản để xem attention.

## Kết luận

Bài viết tổng hợp chuyên sâu bốn cơ chế attention trong LLMs: **Self-Attention**, **MQA**, **GQA**, và **FlashAttention**, với lý thuyết, ví dụ thực tiễn, mục tiêu cuối cùng, và thông tin mã minh họa (trong tài liệu bổ sung).

- **Self-Attention:** Tạo biểu diễn ngữ cảnh phong phú (trả lời câu hỏi tài liệu).
- **MQA:** Tối ưu tốc độ suy luận thời gian thực (chatbot sinh câu chuyện).
- **GQA:** Cân bằng chất lượng và hiệu quả (dịch máy thời gian thực).
- **FlashAttention:** Xử lý chuỗi dài hiệu quả (tóm tắt tài liệu dài).

Tài liệu và notebook đáp ứng yêu cầu đề thi, sẵn sàng nộp. Nếu cần bổ sung, vui lòng liên hệ!